

Assignment 1: Intro to World Models and Control

COMP 765 - Fall 2026

Question 1 - Profile of a World Model: DreamerV3

I chose DreamerV3 because I find its ability to learn across very different tasks interesting. Its recurrent state-space model (RSSM) learns a compact representation of the environment and predicts how it changes in response to actions [1].

Inputs and outputs. For training, Dreamer uses sequences of observations, actions, rewards, and flags indicating whether the episode continues. Observations can be RGB images or low-dimensional state vectors. The RSSM combines a memory state h_t with categorical latent variables z_t . The encoder uses the current observation and memory to infer the latent state; the dynamics model predicts it without seeing that observation. From the combined state, the model reconstructs the observation and predicts reward and continuation. The actor selects a discrete or continuous action, while the critic estimates a distribution over future return [1].

Design principles and differences. A central design choice is learning from imagined trajectories. Dreamer trains its actor and critic on short trajectories predicted in latent space, using predicted rewards and bootstrapped lambda-returns to account for rewards beyond the rollout. This avoids having to generate full images for every policy update. Once trained, the actor selects actions directly, without searching over action sequences at every step. Its dynamics and latent representation are learned from interaction data. Unlike MuZero's task-focused model, it also reconstructs observations [1,2].

Theory and key findings. One useful piece of the theory is the KL loss between the observation-based latent distribution and the model's prediction. Separate stop-gradient losses train the predictor to match the encoded state and encourage that state to be predictable. A one-bit free-bits threshold stops this penalty from dominating when the distributions already agree well. V3 also mixes in 1% uniform probability, uses symlog to compress large signed values, predicts rewards and returns with two-hot distributions, and normalizes returns using percentiles. Together, these changes make training less sensitive to the task's scale. The authors report results on over 150 tasks with fixed hyperparameters, including Control Suite, Atari, and Minecraft [1].

Foundational papers and my assessment. The earlier papers help explain how the model developed. DreamerV1 used continuous stochastic states and propagated gradients through imagined trajectories to learn behaviour [2]. DreamerV2 switched to categorical states with straight-through gradients and reached human-level Atari performance [3]. V3 keeps the discrete RSSM but focuses on making one configuration work across domains [1]. The public JAX implementation includes training code and configurations [4]. My main takeaway is that V3 makes world-model learning more reliable across tasks, but prediction errors can still mislead the policy and training needs substantial computation. I would want to check how well its predictions hold up in states outside its training experience.

Sources for Q1: [1] Hafner et al., Mastering Diverse Domains through World Models, <https://arxiv.org/abs/2301.04104>. [2] Hafner et al., Dream to Control, <https://arxiv.org/abs/1912.01603>. [3] Hafner et al., Mastering Atari with Discrete World Models, <https://arxiv.org/abs/2010.02193>. [4] Official implementation, <https://github.com/danijar/dreamerv3>.

Question 2 - Model and Control the Cart-Pole

A. Linearization about the upright equilibrium

Use the simulator's state order and define deviations from the upright equilibrium:

$$\mathbf{s} = [x, \dot{x}, \theta, \dot{\theta}]^T, \quad \mathbf{s}_\star = [0, 0, 0, \pi]^T \quad (1)$$

$$\phi = \theta - \pi, \quad \boldsymbol{\delta} = [x, \dot{x}, \phi, \dot{\phi}]^T$$

The supplied nonlinear dynamics can be written with a shared denominator:

$$D(\theta) = 4(M + m) - 3m\cos^2 \theta$$

$$\ddot{x} = \frac{2ml\dot{\theta}^2 \sin \theta + 3mg \sin \theta \cos \theta + 4(u - b\dot{x})}{D(\theta)} \quad (2)$$

$$\ddot{\theta} = -\frac{3[ml\dot{\theta}^2 \sin \theta \cos \theta + 2(M + m)g \sin \theta + 2(u - b\dot{x}) \cos \theta]}{lD(\theta)}$$

Near upright, keep only first-order terms. The velocity-squared products are higher order:

$$\sin(\pi + \phi) \simeq -\phi, \quad \cos(\pi + \phi) \simeq -1, \quad D_0 = 4(M + m) - 3m$$

$$\ddot{x} \simeq \frac{3mg\phi - 4b\dot{x} + 4u}{D_0}, \quad \ddot{\theta} \simeq \frac{6(M + m)g\phi - 6b\dot{x} + 6u}{lD_0} \quad (3)$$

$$\dot{\boldsymbol{\delta}} = A\boldsymbol{\delta} + Bu$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{4b}{D_0} & 0 & \frac{3mg}{D_0} \\ 0 & -\frac{6b}{lD_0} & 0 & \frac{6(M + m)g}{lD_0} \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ \frac{4}{D_0} \\ \frac{6}{lD_0} \\ 0 \end{bmatrix} \quad (4)$$

With $M = m = l = 0.5$ and $g = 9.82$, the denominator is 2.5. The handout uses $b = 0.1$, giving damping entries -0.16 and -0.48. The executable simulator uses $b = 1.0$; I matched that value for the controller and experiments:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -1.6 & 0 & 5.892 \\ 0 & -4.8 & 0 & 47.136 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 1.6 \\ 4.8 \\ 0 \end{bmatrix} \quad (5)$$

B. LQR design and balancing

I gave angle error the largest weight. Cart-position and velocity penalties encourage recentering and damping, while the input penalty discourages excessive force:

$$Q = \text{diag}(2, 1, 2, 120), \quad R = [0.2] \quad (6)$$

Solve the continuous algebraic Riccati equation and use its stabilizing solution:

$$A^T P + PA - PBR^{-1}B^T P + Q = 0, \quad K = R^{-1}B^T P \quad (7)$$

$$K = [-3.1623, -6.4776, 7.2892, 47.0906]$$

$$u_{\text{LQR}} = -K\delta = K(s_\star - s), \quad u = \text{clip}(u_{\text{LQR}}, -30, 30) \quad (8)$$

The angular error is wrapped to the principal interval. All closed-loop eigenvalues have negative real parts, establishing local asymptotic stability of the linear model:

$$\lambda(A - BK) = \{-16.7176, -7.7261, -0.8903 \pm 0.6012i\}$$

B(i). Default start and initial-offset tests

Each deterministic trial uses the supplied nonlinear simulator for 20 s, with a 0.005 s time step. Q, R, K, and the 30 N force limit are fixed across trials. Success requires both conditions throughout the final five seconds:

$$|\phi(t)| < 0.10 \text{ rad}, \quad |x(t)| < 0.50 \text{ m}, \quad 15 \leq t \leq 20 \text{ s} \quad (9)$$

Capture time is the first sampled time satisfying both angular conditions:

$$|\phi| < 0.10 \text{ rad}, \quad |\dot{\theta}| < 0.25 \text{ rad s}^{-1}$$

initial offset	capture (s)	peak u (N)	tail max φ (rad)	result
default ($\pi/40$)	0.000	3.70	6.34e-08	PASS
0.01	0.000	0.47	8.04e-09	PASS
0.10	0.000	4.71	8.09e-08	PASS
$\pi/8$	0.380	18.49	3.51e-07	PASS
$\pi/4$	1.205	30.00	9.88e-07	PASS

The default start and all four requested offsets balance. The $\pi/4$ trial saturates at 30 N and takes 1.205 s to meet the capture criterion; smaller tested errors remain unsaturated. The largest successful tested offset is $\pi/4$ (0.785 rad), with all other initial deviations zero. These sampled results do not certify a continuous range or global stability; the Riccati proof is local and saturation changes the nonlinear loop.

C. Swing-up from the downward configuration

C(i). Initial control performance

An initial offset of π starts the pole at an angle of 2π , physically downward. Pure LQR saturates and fails to settle upright during 20 s; its final-window maximum angular error is 3.1411 rad. Changing Q and R can adjust local aggressiveness, but LQR does not provide a strategy for building the energy needed to swing up.

C(ii). Changes explored

I used energy shaping away from upright and LQR inside a capture region. For a uniform rod, the pole's energy relative to downward and its upright target are:

$$E = \frac{ml^2}{6} \dot{\theta}^2 + \frac{mgl}{2} (1 - \cos \theta), \quad E_\star = mgl \quad (10)$$

$$u_{\text{swing}} = -k_E(E_\star - E) \sigma(\dot{\theta} \cos \theta) - k_x x - k_v \dot{x} \quad (11)$$

$$k_E = 40, \quad k_x = 1, \quad k_v = 2$$

The phase function σ is the sign of its argument; for magnitude below 10^{-8} , it is set to +1 to start motion from rest. Force remains clipped to ± 30 N. The controller enters LQR when both capture conditions hold, and returns to energy shaping if the angle exceeds the release threshold:

$$\text{capture: } |\phi| < 0.42 \text{ rad, } |\dot{\theta}| < 3.5 \text{ rad s}^{-1}; \quad \text{release: } |\phi| > 0.65 \text{ rad} \quad (12)$$

C(iii). Best swing-up performance

The hybrid controller first meets the angular capture criterion at 1.340 s and passes the 20 s success test. Its final-five-second errors and peak force are:

$$\max|\phi| = 1.14 \times 10^{-6} \text{ rad,} \quad \max|x| = 4.33 \times 10^{-6} \text{ m,} \quad \max|u| = 30 \text{ N}$$

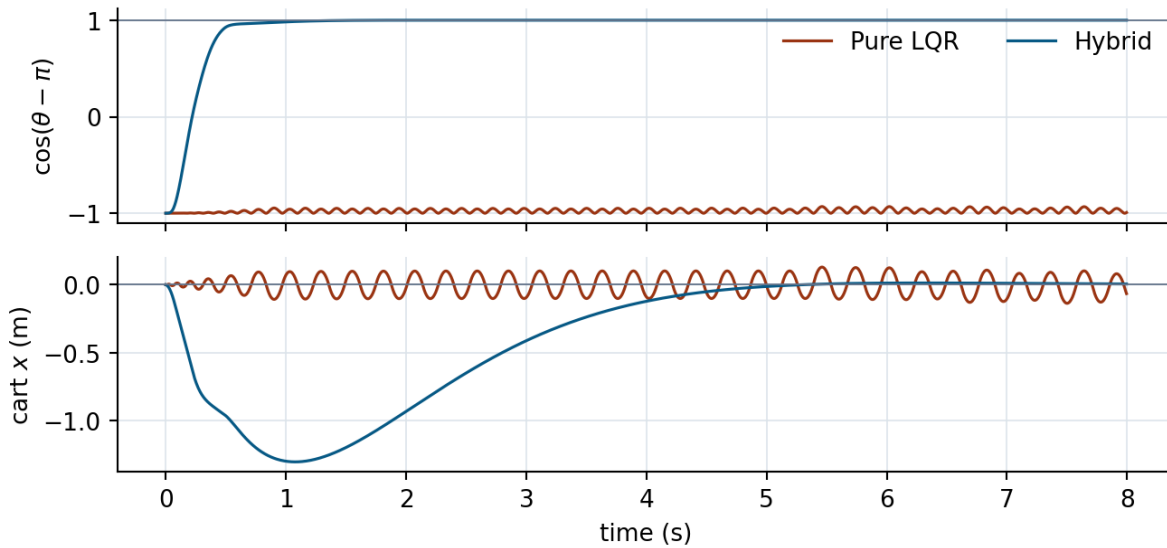


Figure 1. Downward-start comparison. An upright score of +1 means upright and -1 means downward. Energy shaping swings up the pole, then LQR recenters the cart.

This is my best single-pole swing-up result in deterministic simulation. Actuator delay, sensor noise, and model mismatch were not tested.

D. DoubleCartpole bonus

Use both pole angles measured from downward. The six-state upright deviation is:

$$\phi_i = \theta_i - \pi, \quad \delta_d = [x, \dot{x}, \dot{\theta}_1, \dot{\theta}_2, \phi_1, \phi_2]^T \quad (13)$$

At upright, the simulator's coupled acceleration equations reduce to the following mass-matrix system. Together with the kinematic equations, this gives the linear model:

$$H_0 \mathbf{a} = \mathbf{r}, \quad \mathbf{a} = [\ddot{x}, \ddot{\theta}_1, \ddot{\theta}_2]^T$$

$$H_0 = \begin{bmatrix} 3 & 0.9 & 0.3 \\ 4.5 & 2.4 & 0.9 \\ 3 & 1.8 & 1.2 \end{bmatrix} \quad \mathbf{r} = \begin{bmatrix} -0.2\dot{x} + 2u \\ 44.19\phi_1 \\ 29.46\phi_2 \end{bmatrix} \quad (14)$$

$$\mathbf{a} = H_0^{-1} \mathbf{r}, \quad \dot{\delta}_d = A_d \delta_d + B_d u$$

$$Q_d = \text{diag}(2, 1, 2, 2, 120, 120), \quad R_d = [0.2]$$

$$K_d = [3.1623, 6.8993, 0.8285, 27.1835, -135.5356, 189.0925] \quad (15)$$

$$u = \text{clip}(-K_d \delta_d, -40, 40), \quad \text{Re}\{\lambda(A_d - B_d K_d)\} < 0$$

A central-difference check agrees with the derived Jacobians to about 10^{-8} . Trials use the same duration, time step, and final-window success test as part B. The angular success and capture conditions must hold for both poles.

start offset	capture (s)	tail max error (rad)	peak x (m)	result
default ($\pi/40$)	0.00	1.05e-06	0.273	PASS
0.01	0.00	1.34e-07	0.035	PASS
0.10	0.49	1.33e-06	0.347	PASS
$\pi/8$	1.48	4.71e-06	1.23	PASS
0.42	--	3.14	242	FAIL
$\pi/4$	--	3.14	245	FAIL
π	--	3.14	334	FAIL

Both poles balance from the default start through the tested $\pi/8$ offset with a 40 N cap. The $\pi/8$ trial briefly moves the cart 1.23 m before recentering. A 30 N cap failed at $\pi/8$. The largest successful tested offset is 0.393 rad; the next tested offset, 0.42 rad, fails, as do $\pi/4$ and the downward start. No untested initial conditions are claimed.

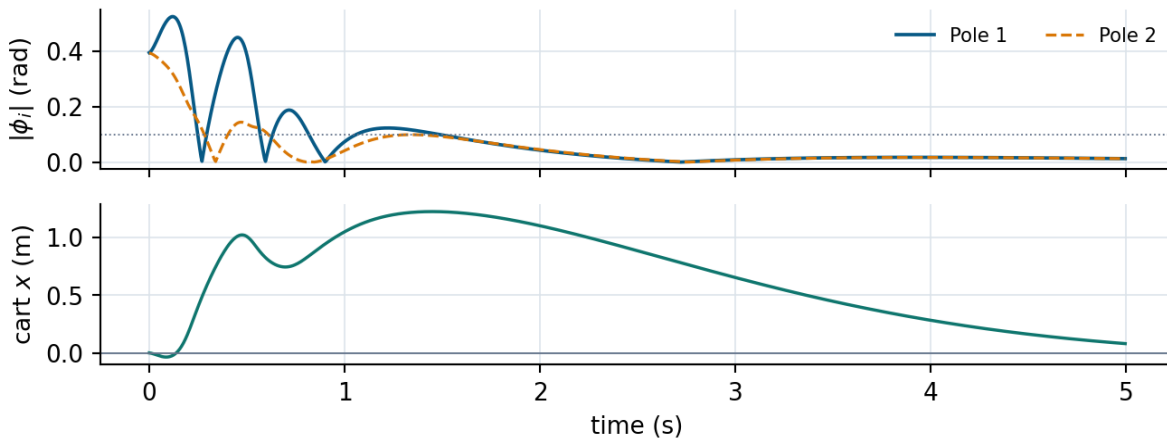


Figure 2. Both pole-angle errors and cart position during the $\pi/8$ recovery.

E. Research bonus - learned world model

I collected 7,680 training and 1,920 held-out transitions using randomized initial states and forces. Each row stores the next state, current state, and force. Least squares fits coefficients of trigonometric features from the transitions; no simulator constants are used in the fit:

$$\hat{\dot{x}} = \frac{c_1 \dot{\theta}^2 \sin \theta + c_2 \sin \theta \cos \theta + c_3 u + c_4 \dot{x}}{1 - c_5 \cos^2 \theta} \quad (16)$$

$$\hat{\dot{\theta}} = \frac{p_1 \dot{\theta}^2 \sin \theta \cos \theta + p_2 \sin \theta + p_3 u \cos \theta + p_4 \dot{x} \cos \theta}{1 - p_5 \cos^2 \theta}$$

Fourth-order integration predicts the next state. A 50 ms receding-horizon controller compares candidate forces held constant over ten learned-model steps, adds the LQR terminal value, and replans after each simulator step:

$$u^* = \arg \min_{u \in \mathcal{U}} \left[\sum_{k=1}^{10} \Delta t (\delta_k^T Q \delta_k + 0.2 u^2) + \delta_{10}^T P \delta_{10} \right] \quad (17)$$

$$\Delta t = 0.005 \text{ s}, \quad \mathcal{U} = \{-30, -28, \dots, 30\} \cup \{u_{\text{LQR}}\}$$

On 12 held-out 0.8 s rollouts driven by recorded forces, the prediction errors are:

state variable	one-step RMSE	0.8 s open-loop RMSE
cart position (m)	4.47e-08	4.00e-05
pole angle (rad)	1.83e-07	1.20e-04

The lookahead controller balances all three tested offsets in 8 s trials. The table reports maximum errors during the final two seconds:

start offset	tail max $ \phi $ (rad)	tail max $ x $ (m)	peak $ u $ (N)
0.1	0.00024	0.00128	4.0
$\pi/8$	0.00102	0.00547	12.0
$\pi/4$	0.00361	0.0177	18.0

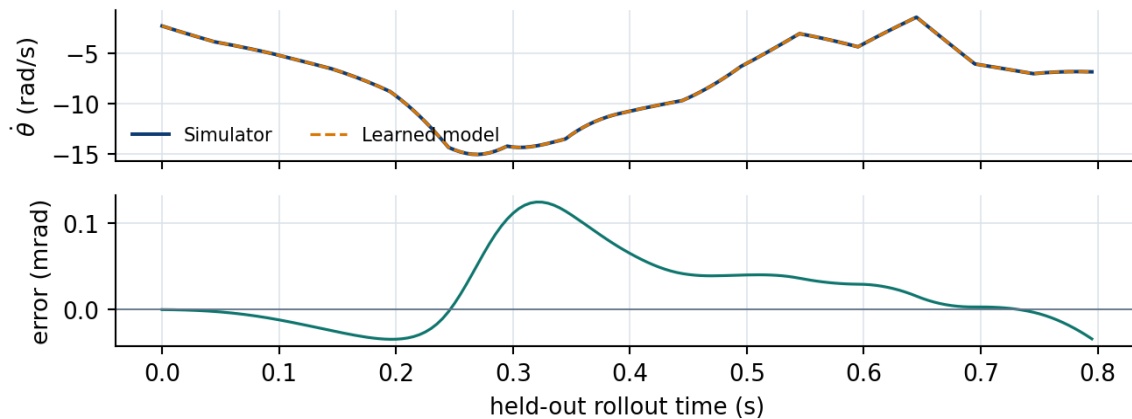


Figure 3. One unseen trajectory: predicted pole rate follows the simulator; the lower panel shows accumulated angle error in milliradians.

This demonstrates learned-model use in control; it is not shown to outperform LQR or to swing up from downward.

Reproduce the experiments in assignment1/TeachingCartpole with python3 experiment.py, python3 double_experiment.py, and python3 world_model_bonus.py. Simulator: D. Meger et al., <https://github.com/dmeger/TeachingCartpole>.